Deploying the AIMScribe Server to DigitalOcean

End to end: the droplet, the stack, the endpoints, and what to tell CMED · AIMS LAB · 9 October 2026

Everything below is taken from the stack as it stands. The whole server — nine services, both databases, the gateway, the archive worker — moves to one DigitalOcean droplet. **The recorders and CMED's page do not change in any way other than one address each.**

	Stage	Time	Who
A	**The endpoint status table** — what works before and after. Give this to CMED	read it now	you → CMED
B	What to have in hand before starting	15 min	AIMS LAB
C	**The deployment, twelve steps**	**2–3 hours**	AIMS LAB
D	Proving every endpoint answers	15 min	AIMS LAB
E	Moving the recorders onto the new address	30 min	AIMS LAB
F	Backups, monitoring, rollback	30 min	AIMS LAB

The one thing to understand before starting
CMED's page never learns the server address. It talks to the recorder on the same PC, over loopback. Only **CMED's own server** and **each recorder** hold the AIMS LAB address, which is why this deployment changes exactly two things on the outside — one line in CMED's environment, and one line in each recorder's configuration. Nothing is rebuilt and no code changes.

PART A

The endpoint status table

What works today, what works after deployment - the page to send CMED

1 · Every endpoint, before and after

This is the table to put in front of CMED. "Today" means the system as it runs now, on a PC on a private network. "After deployment" means once Part C is done.

	Endpoint	Today	After deployment	Why
1	`GET https://<host>/health`	**NO**	**YES**	Needs a public name and a certificate. Both arrive with the droplet
2	`GET https://<host>/`	**NO**	**YES**	same
3	`POST https://<host>/api/v2/clinical/patient-information` *API 2*	**NO**	**YES**	**This is the one CMED is waiting for.** The code is finished and tested; it has had no address to answer on
4	`POST https://<host>/api/v2/clinical/prescription` *API 3, Channel B*	**NO**	**YES**	same as 3
5	`ws://127.0.0.1:5050/ws` — `start` *API 1*	**YES**	**YES**	**Unaffected.** Loopback on the clinic PC — it never involved our server
6	`ws://127.0.0.1:5050/ws` — `prescription_built` *API 3, Channel A*	**YES**	**YES**	Unaffected
7	`ws://127.0.0.1:5050/ws` — `doctors`	**YES**	**YES**	Unaffected
8	`ws://127.0.0.1:5050/ws` — `status`	**YES**	**YES**	Unaffected

Read the shape of that table. The four local ones already work and are untouched by the deployment. The four server ones are all blocked by the same single cause — no public address — and all four are unblocked by the same single act. There is no code to write on our side.

2 · What to say to CMED, in writing

A note you can send as it stands:

```
The four endpoints your page uses on the clinic PC work today and are not
affected by our deployment. Your front-end team can build and test against
them now - install the recorder on one PC and they are live:

    ws://127.0.0.1:5050/ws   commands: start, prescription_built,
                                       doctors, status

The two endpoints your backend uses are finished and tested but have no
public address yet, because our server is currently on a private network.
They will answer as soon as our DigitalOcean deployment is done:

    POST  https://<host>/api/v2/clinical/patient-information    (API 2)
    POST  https://<host>/api/v2/clinical/prescription           (API 3b)
    GET   https://<host>/health                 - no key, to check we are up

Nothing about the message format, the reply codes, the key or the limits
changes between now and then. Your backend can be written against the
specification today and pointed at the host when we send it.
```

```
Please hold the host as configuration, not a compiled-in constant: we will
give you a staging host first and a production host later.
```

PART B

Before you start

Fifteen minutes of preparation that saves an afternoon

3 · What you need in hand

	What	Where from	Note
■	A DigitalOcean account with billing	digitalocean.com	about $115/month for droplet and backups
■	**A domain name you control**	a UIU subdomain, or a registrar	**Required.** A certificate cannot be issued for an IP address
■	A Cloudflare account with an **R2 bucket**	cloudflare.com	3 TB of audio. Create it before the server needs it
■	The R2 access key and secret	Cloudflare → R2 → Manage API tokens	Shown once
■	An SSH key pair	your own machine	Password login will be turned off
■	The project code	the AIMScribe_v3 repository	cloned on the droplet
■	CMED's clinic codes	CMED	**needed at step 10**, not before
■	An email for the certificate	yours	Let's Encrypt expiry notices

The domain is the only hard prerequisite. Everything else can be done as you go, but the gateway asks Let's Encrypt for a certificate the first time it starts, and that requires a name already pointing at the droplet. Decide the name before you begin.

4 · What will be running when you are done

Service	What it does	Exposed?
gateway (Caddy)	HTTPS, the certificate, security headers, a 2 MB body cap	**yes — 80 and 443**
api	every endpoint, on internal port 6000	no
postgres	both databases	no
pgbouncer	connection pooling	no
redis	the work queue	no
worker	background processing	no
archive-worker	joins recordings, uploads to R2, verifies	no
backup	nightly encrypted database dump	no
monitor	watches disk, queue depth, unconfirmed recordings	no

Only the gateway is reachable from outside, on ports 80 and 443 only. Every other service talks on a private Docker network. The API itself is never published — which is why there is no way to reach the recorder endpoints or the dashboard except through the gateway.

PART C

The deployment

Twelve steps, two to three hours, done once

Step 1 · Create the droplet

Field	Choose
Region	**Bangalore / BLR1** — lowest latency to Dhaka, 45–70 ms
Image	**Ubuntu 24.04 (LTS) x64**
Type	**Basic** → **Regular (SSD)**
Size	**8 vCPU / 16 GB / 320 GB —** `s-8vcpu-16gb` **— $96/mo**
Authentication	**SSH key.** Not a password
Backups	**Enable** — +20%, about $19/mo
Monitoring	Enable — free
Hostname	`aimscribe-prod-blr1`

Note the public IPv4 address when it finishes.

Step 2 · Point the name at it

```
A     aimscribe.uiu.ac.bd        -> <the droplet IPv4>      TTL 300

# then, from your own machine, wait until this answers with that address:
nslookup aimscribe.uiu.ac.bd
```

Do not go further until the name resolves. Caddy will ask Let's Encrypt for a certificate on first start, and if the name does not yet point here the request fails and is rate-limited for a while.

Step 3 · The firewall

In the DigitalOcean control panel, Networking → Firewalls, create one and attach it to the droplet:

Direction	Protocol	Port	Source / destination	Why
Inbound	TCP	**22**	**your own IP only**	administration
Inbound	TCP	**80**	all IPv4, all IPv6	certificate issue and renewal
Inbound	TCP	**443**	all IPv4, all IPv6	**the API and the gateway**
Inbound	UDP	**443**	all IPv4, all IPv6	HTTP/3
Outbound	all	all	all	R2 uploads, updates, certificates

Nothing else. No database port, no Redis port, no API port. If you ever find yourself opening 5432 or 6379 to the internet, stop — use an SSH tunnel instead.

Step 4 · Harden the machine and install Docker

```
ssh root@<droplet-ip>

# a working user
adduser --disabled-password --gecos "" aims
usermod -aG sudo aims
rsync --archive --chown=aims:aims ~/.ssh /home/aims

# no root login, no passwords
sed -i "s/^#*PermitRootLogin.*/PermitRootLogin no/" /etc/ssh/sshd_config
sed -i "s/^#*PasswordAuthentication.*/PasswordAuthentication no/" \
    /etc/ssh/sshd_config
systemctl restart ssh

# updates, and automatic security patches
apt update && apt upgrade -y
apt install -y unattended-upgrades fail2ban git curl
dpkg-reconfigure -plow unattended-upgrades

# the clock - recordings are filed by the clinic's local date
timedatectl set-timezone Asia/Dhaka
timedatectl set-ntp true

# Docker
curl -fsSL https://get.docker.com | sh
usermod -aG docker aims
systemctl enable --now docker
```

The timezone line is not cosmetic. A recording is filed under the clinic's local date, decided once when the session closes. A server on UTC files the evening's consultations under tomorrow.

Step 5 · Get the code

```
exit                    # back out of root
ssh aims@<droplet-ip>

git clone https://github.com/SakibZaman021/AIMScribe_v3.git
cd AIMScribe_v3/deploy/uiu

# the folder is named uiu for historical reasons; it is the SERVER stack,
# and it is the right one for this droplet. deploy/local is the dev stack.
ls   # docker-compose.yml  Caddyfile  .env.example  postgres/  backup/ ...
```

Step 6 · The settings file

Eleven values have no default and the stack refuses to start without them, plus `AIMS_ADMIN_KEY`, which you need in step 10. **Generate the secrets on the server; do not invent them and do not reuse any.**

```
cp .env.example .env
chmod 600 .env

# five passwords and two service keys
for v in POSTGRES_SUPERUSER_PASSWORD AIMS_RECORDINGS_PASSWORD \
         AIMS_CLINICAL_PASSWORD AIMS_MONITOR_PASSWORD REDIS_PASSWORD \
         AIMS_WORKER_KEY AIMS_ADMIN_KEY; do
  echo "$v=$(openssl rand -base64 36 | tr -d '/+=' | head -c 40)" >> .env
done

# and the two encryption keys (32 bytes each)
echo "AIMS_BACKUP_KEY=$(openssl rand -base64 32)" >> .env
echo "AIMS_COPY_KEY=$(openssl rand -base64 32)"   >> .env
```

The eleven the stack enforces	What it is
`POSTGRES_SUPERUSER_PASSWORD`	generated above
`AIMS_RECORDINGS_PASSWORD`	the `aims_recordings` database
`AIMS_CLINICAL_PASSWORD`	the `aims_clinical` database

`AIMS_MONITOR_PASSWORD`	read-only, for the monitor
`REDIS_PASSWORD`	the queue
`AIMS_WORKER_KEY`	the archive worker's own key
`AIMS_PUBLIC_HOST`	**the name from step 2** — the certificate is issued for it
`AIMS_ACME_EMAIL`	**yours** — Let's Encrypt expiry notices
`AIMS_DB_PATH`	`/srv/aimscribe/db`
`AIMS_ARCHIVE_PATH`	`/srv/aimscribe/archive`
`AIMS_BACKUP_PATH`	`/srv/aimscribe/backup`

Then edit `.env` and set the rest by hand:

```
AIMS_PUBLIC_HOST=aimscribe.uiu.ac.bd        # the name from step 2
AIMS_ACME_EMAIL=you@uiu.ac.bd               # certificate notices
TZ=Asia/Dhaka

AIMS_DB_PATH=/srv/aimscribe/db              # the databases
AIMS_ARCHIVE_PATH=/srv/aimscribe/archive    # recordings in transit
AIMS_BACKUP_PATH=/srv/aimscribe/backup      # nightly dumps

# Cloudflare R2 - where the audio actually lives
AIMS_COPY_ENDPOINT=https://<account-id>.r2.cloudflarestorage.com
AIMS_COPY_BUCKET=aimscribe-audio
AIMS_COPY_ACCESS_KEY=<from Cloudflare>
AIMS_COPY_SECRET_KEY=<from Cloudflare>
```

Write AIMS_BACKUP_KEY and AIMS_COPY_KEY down somewhere safe, off this machine, now

These two keys encrypt the database backups and every audio file before it leaves for Cloudflare. **If the droplet is lost and these keys were only on it, the backups and the entire cloud archive are permanently unreadable.** Put them in the access sheet, and keep a copy somewhere that is not this server.

Step 7 · Create the folders and the R2 bucket

```
sudo mkdir -p /srv/aimscribe/{db,archive,backup}
sudo chown -R aims:aims /srv/aimscribe
df -h /srv            # confirm the 320 GB is there
```

In Cloudflare, R2 → Create bucket:

Setting	Value
Name	`aimscribe-audio`
Location	Asia-Pacific
Object versioning	**On** — so a mistaken delete can be undone
Public access	**Off.** Never public

Step 8 · Start it

```
docker compose up -d

# the databases, their roles and all eleven migrations run themselves
docker compose logs -f api | head -60

# all nine should be healthy or running
docker compose ps
```

Give the gateway a minute. On first start it asks Let's Encrypt for the certificate; until that completes HTTPS will refuse connections. Watch it with `docker compose logs -f gateway` — you are looking for `certificate obtained successfully`.

Step 9 · Prove it answers — from your own machine, not the server

```
curl -s https://aimscribe.uiu.ac.bd/health | jq .

# expect:
#   "status":   "healthy"
#   "database": "connected"
#   "redis":    "connected"
#   "system":   "AIMScribe v3 - AIMS LAB server"
#                 ^ check it STARTS WITH "AIMScribe v3"
```

Run it from somewhere else, not from the droplet. Testing from the server proves the program is running; it does not prove DNS, the firewall, the certificate or the route work — which is the whole point of this step.

Step 10 · Register the clinics, the doctors and CMED's key

These are admin API calls, authenticated with the `X-Admin-Key` you generated in step 6. Run them from the droplet.

```
A=$(grep ^AIMS_ADMIN_KEY= .env | cut -d= -f2)
H=https://aimscribe.uiu.ac.bd/api/v2

# one clinic. Note cmed_hospital_id - THIS is the mapping to CMED's own
# code, and it is what stops a visit being orphaned (see below).
curl -s -X POST $H/admin/hospital -H "X-Admin-Key: $A" \
  -H "Content-Type: application/json" \
  -d '{ "hospital_id": "AALO_DHOLPUR",
        "name": "Aalo Clinic, Dholpur",
        "timezone": "Asia/Dhaka",
        "cmed_hospital_id": "<CMED's code for this clinic>" }'

# repeat for AALO_KARAIL, AALO_MIRPUR, AALO_SHYAMPUR,
#            AALO_NARAYANGANJ, AALO_ERSHADNAGAR, AMADER_SUSASTHO

# the doctors at each, so CMED's selector can be filled from /doctors
curl -s -X POST $H/admin/doctor -H "X-Admin-Key: $A" \
  -H "Content-Type: application/json" \
  -d '{ "doctor_id": "DR0042", "hospital_id": "AALO_DHOLPUR",
```

```
        "full_name": "Dr Rahman", "active": true }'

# CMED's Channel B key - PRINTED ONCE, NEVER RECOVERABLE
curl -s -X POST $H/admin/cmed-key -H "X-Admin-Key: $A"
```

Capture that key the moment it prints and send it to CMED over a channel that is not email if you can. It cannot be retrieved afterwards — only revoked and reissued, with `POST /admin/cmed-key/revoke`.

`cmed_hospital_id` is the blocking prerequisite, and this is where it goes

It is the field that maps **CMED's clinic code** to ours. Without it a visit CMED describes cannot be tied to the clinic it happened in: the record is stored, **CMED receives a normal 202**, and the recording is erased after twenty-four hours — silently, from their side.

So get CMED's seven codes before this step. If they are not available yet, register the clinics anyway and fill the field in later with the same call; it is an update, not a re-registration.

Step 11 · Enrol the clinic PCs against the new address

This is the step people underestimate. **A recorder is enrolled to one clinic and one server address, and the enrolment is signed** — so pointing a recorder at a new server is not simply an edit to a settings file.

	The four places the address lives	Changes?
1	**The server:** `AIMS_PUBLIC_HOST` in `deploy/uiu/.env`	**yes** — step 6
2	The archive worker	**no** — it reaches the API inside the machine
3	**CMED:** `AIMS_SERVER_URL` in CMED's own server environment	**yes** — tell them
4	**Every recorder:** `AIMS_BACKEND_URL` in the `.env` beside the exe	**yes** — per PC
—	**CMED's web page**	**no — it never knew the address**

For each clinic PC: issue a fresh single-use enrolment token on the server, then run the installer or the enrolment script on that PC with the new address. **Do one PC first and record a full test consultation on it before touching the other thirteen.**

```
# The fleet is minted from a CSV - one row per LAPTOP, not per doctor.
# It creates any hospital that does not exist, mints one single-use token
# per machine, and writes a one-page instruction sheet for each.

#   backend/scripts/laptops.csv
#   hospital_id,hospital_name,room,doctor_id,doctor_name
#   AALO_DHOLPUR,"Aalo Clinic, Dholpur",Room 3,,
#   AALO_DHOLPUR,"Aalo Clinic, Dholpur",Room 4,,

docker compose exec api python scripts/mint_enrolment_tokens.py \
    scripts/laptops.csv

# then on each clinic PC, in the folder beside AIMScribe_Agent.exe:
#   AIMS_BACKEND_URL=https://aimscribe.uiu.ac.bd
# enrol with that machine's token, and restart the agent
```

Two things about `laptops.csv`

Add rows, never rewrite the file. The script creates any hospital it finds and the register is cumulative — a trimmed CSV reads as clinics that no longer exist.
One row per laptop, not per doctor. A token binds a *machine* to a *clinic*, and that is all it decides. The doctor arrives with each consultation from CMED, so one laptop serves the morning and afternoon shifts untouched.

Step 12 · Backups and the restore drill

```
# the backup service already runs nightly. Prove it works TODAY:
docker compose exec backup /backup/backup.sh
ls -lh /srv/aimscribe/backup

# then put it back into a scratch database and count the rows
docker compose exec backup /backup/restore.sh --help

# the full drill, both databases and a day of audio, is written up in
#   deploy/uiu/README.md, section 7
```

A backup nobody has restored is not a backup. Do the restore drill on day one, then once a month. This is the single most valuable half hour in the whole deployment.

PART D

Proving every endpoint

The checks to run after Part C, and to send CMED as evidence

5 · The eight checks

	Check	Command or action	Pass
1	The name resolves	`nslookup <host>`	the droplet's IP
2	The certificate is real	`curl -sI https://<host>/health`	no TLS warning
3	**Health, from outside**	`curl -s https://<host>/health`	`status: healthy`
4	It is the right system	the same reply	`system` **starts with** `AIMScribe v3`
5	The key is enforced	POST API 2 with no key	**401**
6	**API 2 accepts**	POST API 2 with the key	**202**
7	Retries are safe	POST the identical body again	**200** ALREADY_RECEIVED
8	**API 3 Channel B accepts**	POST the prescription	**202**

Our script does checks 3 to 8 in one command: `python tools/channel_b_test.py --server https://<host> --key <key>`. Run it, keep the output, and send it to CMED — it is the evidence that the gateway is ready for them.

And then the one check that actually matters

Checks 1 to 8 prove the gateway works. **They do not prove the integration works.** Record one real consultation on one clinic PC, with CMED sending all four messages, and then **confirm on the dashboard that the recording was matched and archived**. Every check above can pass while the five identifying fields silently fail to match — in which case the recording is deleted twenty-four hours later and nobody notices.

6 · Where things are on the droplet

What	Path	Note
The code and the stack	`/home/aims/AIMScribe_v3/deploy/uiu`	where you run docker compose
The settings and the keys	`…/deploy/uiu/.env`	**mode 600. Never commit it**
The databases	`/srv/aimscribe/db`	about 10 GB, growing
Recordings in transit	`/srv/aimscribe/archive`	**clinic / doctor / date / consultation folder**
Nightly backups	`/srv/aimscribe/backup`	encrypted
The permanent audio	**Cloudflare R2, not this machine**	`copies/clinic/doctor/date/`

PART F

Keeping it running, and going back

What to watch, and what to do if the deployment has to be undone

7 · What to watch

What	How	Act when
The system is up	`/health`, every minute, from outside	**not healthy twice in a row**
Disk	DigitalOcean monitoring, or `df -h`	**above 80%**
Unconfirmed recordings	the dashboard, every morning	**anything above zero**
The upload buffer	the monitor service	above half full
Backups	that last night's file exists	any night it does not
The certificate	renews itself, every 60 days	only if /health goes unreachable

8 · Routine operations

```
cd ~/AIMScribe_v3/deploy/uiu

docker compose ps                       # what is running
docker compose logs -f api              # follow the API
docker compose logs --tail 200 gateway # the certificate, the requests
docker compose restart api              # restart one service

# update to a new release, outside clinic hours
git pull && docker compose up -d --build

# a consultation in progress survives an API restart: the recorder holds
# its pieces and retries. Do it between patients anyway.
```

9 · If it has to be undone

	Situation	What to do
1	The deployment fails before any clinic uses it	**Nothing to undo.** Destroy the droplet. The old PC stack is untouched and still has every recording
2	It works, but a problem appears on day one	Point the recorders' `AIMS_BACKEND_URL` back, and tell CMED to revert `AIMS_SERVER_URL`. **Keep the droplet running** — it holds recordings the old one does not
3	The droplet is lost entirely	Rebuild from the DigitalOcean backup, or from scratch with this runbook plus the nightly database backup. **The audio is in R2 and is unaffected**

Which is the real reason to put the audio in R2 rather than on the droplet. Losing the server costs you an afternoon of rebuilding. Losing the server with 2 TB of consultations on its disk would cost the study.

10 · One page to remember

Question	Answer
Which compose file?	`deploy/uiu/docker-compose.yml` — the server stack, despite the folder name
Which droplet?	`s-8vcpu-16gb`**, Bangalore**, Ubuntu 24.04, backups on
What must exist first?	**A domain name pointing at the droplet.** The certificate depends on it
Which ports open?	**80 and 443 only**, plus SSH from your own IP

How many settings have no default?	**Eleven**, plus `AIMS_ADMIN_KEY` for step 10
Where do CMED's clinic codes go?	`cmed_hospital_id` on each hospital registration. Step 10
Which two secrets must leave the machine?	`AIMS_BACKUP_KEY` **and** `AIMS_COPY_KEY` — without them the backups and the whole R2 archive are unreadable
Where does the audio live?	**Cloudflare R2**, as encrypted WAV. Not on the droplet
What does CMED change?	**One line** — `AIMS_SERVER_URL`. Their page changes nothing
What does each recorder change?	**One line** — `AIMS_BACKEND_URL` — plus a fresh enrolment token
Which endpoints start working?	**All four server ones.** The four local ones already worked. Part A
First thing after it is up?	`curl https://<host>/health` **from somewhere else**
Most valuable half hour?	**The restore drill.** Step 12

Prepared by AIMS LAB, United International University. Companions: the Integration & Test Guide for CMED DevOps, and the Complete Cloud Plan. Every port, path, service and variable here is taken from the stack itself.